Networks Assignment - 01
Lab Report

Question 1

Protocols used in Youtube-uploading video are provided in the form of screenshots of
Statistics->Protocol Hierarchy.

From Ethernet in Lab:

From Mobile Hotspot in Hostel:

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	18696	100.0	19366590	1,084 k	0	0	0	18696
Ethernet	100.0	18696	1.4	275245	15 k	0	0	0	18696
Media Independent Network Transport	0.1	12	0.0	840	47	12	840	47	12
Logical-Link Control	0.1	20	0.0	1422	79	0	0	0	20
HP Extended Local-Link Control	0.1	12	0.0	84	4	0	0	0	12
HP Switch Protocol	0.1	12	0.0	1030	57	12	1030	57	12
Dynamic Trunk Protocol	0.0	8	0.0	208	11	8	208	11	8
Link Layer Discovery Protocol	0.0	5	0.0	1255	70	5	1255	70	5
Internet Protocol Version 4	95.8	17914	1.8	358280	20 k	0	0	0	17914
User Datagram Protocol	88.2	16497	0.7	131976	7,393	0	0	0	16497
Simple Service Discovery Protocol	1.6	292	0.3	55607	3,115	292	55607	3,115	292
QUIC IETF	86.3	16136	92.2	17859512	1,000 k	16136	17846065	999 k	16154
Network Time Protocol	0.0	2	0.0	96	5	2	96	5	2
NetBIOS Name Service	0.0	7	0.0	350	19	7	350	19	7
NetBIOS Datagram Service	0.0	1	0.0	201	11	0	0	0	1
SMB (Server Message Block Protocol)	0.0	1	0.0	119	6	0	0	0	1
SMB MailSlot Protocol	0.0	1	0.0	25	1	0	0	0	1
Microsoft Windows Browser Protocol	0.0	1	0.0	33	1	1	33	1	1
Multicast Domain Name System	0.1	18	0.0	4525	253	18	4525	253	18
Link-local Multicast Name Resolution	0.0	1	0.0	33	1	1	33	1	1
Dynamic Host Configuration Protocol	0.0	5	0.0	1800	100	5	1800	100	5
Domain Name System	0.1	16	0.0	2406	134	16	2406	134	16
Data	0.1	19	0.0	9308	521	19	9308	521	19
Transmission Control Protocol	7.6	1416	3.3	648619	36 k	692	61932	3,469	1416
Transport Layer Security	3.9	721	3.1	603535	33 k	721	598279	33 k	723
Hypertext Transfer Protocol	0.0	3	0.0	361	20	3	361	20	3
Data	0.0	1	0.0	799	44	1	799	44	1
Address Resolution Protocol	4.0	745	0.2	34234	1,917	745	34234	1,917	745
Cisco ISL	0.0	4	0.0	104	5	0	0	0	4

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	17570	100.0	19230564	1,134 k	0	0	0	17570
Ethernet	100.0	17570	1.3	245980	14 k	0	0	0	17570
Internet Protocol Version 4	100.0	17570	1.8	351400	20 k	0	0	0	17570
User Datagram Protocol	91.9	16154	0.7	129232	7,626	0	0	0	16154
QUIC IETF	91.8	16136	92.9	17859512	1,053 k	16136	17846065	1,053 k	16154
Network Time Protocol	0.0	2	0.0	96	5	2	96	5	2
Domain Name System	0.1	16	0.0	2406	141	16	2406	141	16
Transmission Control Protocol	8.1	1416	3.4	648619	38 k	692	61932	3,654	1416
Transport Layer Security	4.1	721	3.1	603535	35 k	721	598279	35 k	723
Hypertext Transfer Protocol	0.0	3	0.0	361	21	3	361	21	3

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	19726	100.0	19875917	890 k	0	0	0	19726
Ethernet	100.0	19726	1.4	276164	12 k	0	0	0	19726
Internet Protocol Version 6	96.5	19038	3.8	761520	34 k	0	0	0	19038
User Datagram Protocol	93.9	18515	0.7	148120	6632	0	0	0	18515
QUIC IETF	1.5	304	0.6	125725	5630	304	114666	5134	326
Multicast Domain Name System	0.0	6	0.0	829	37	6	829	37	6
HiPerConTracer Trace Service	0.0	1	0.0	1225	54	1	1225	54	1
Domain Name System	0.1	19	0.0	2143	95	19	2143	95	19
Data	92.2	18185	91.3	18153168	812 k	18185	18153168	812 k	18185
Transmission Control Protocol	2.6	504	0.1	11680	523	404	9680	433	504
Transport Layer Security	0.5	96	0.7	148472	6648	96	146971	6581	97
Data	0.0	4	0.0	4	0	4	4	0	4
Internet Control Message Protocol v6	0.1	19	0.0	640	28	19	640	28	19
Internet Protocol Version 4	3.4	674	0.1	13480	603	0	0	0	674
User Datagram Protocol	0.6	117	0.0	936	41	0	0	0	117
Multicast Domain Name System	0.0	6	0.0	829	37	6	829	37	6
Dropbox LAN sync Discovery Protocol	0.0	6	0.0	804	36	6	804	36	6
Domain Name System	0.5	105	0.0	9468	423	105	9468	423	105
Transmission Control Protocol	2.8	557	0.1	12044	539	365	8204	367	557
Transport Layer Security	1.0	192	1.0	190745	8541	192	164565	7369	197
Address Resolution Protocol	0.1	14	0.0	392	17	14	392	17	14

Protocols used by applications:

PACKET FORMAT

1) Application Layer:

a) HTTP/HTTPS (Hypertext Transfer Protocol / Hypertext Transfer Protocol Secure)

HTTP (Hypertext Transfer Protocol)

HTTP is the original protocol for data communication on the web. It's a simple, stateless protocol that allows clients to request resources from a server.

- **Usage**: It's used for transmitting unencrypted data, such as basic web pages and images. Because there's no encryption, it's not suitable for sensitive information like login credentials or credit card numbers.
- **Port**: HTTP operates on port **80**.
- **Security**: Data is sent in **plaintext**, meaning it can be intercepted and read by anyone with access to the network connection. This makes it vulnerable to "man-in-the-middle" attacks where an attacker can eavesdrop on, or even alter, the communication.

HTTPS (Hypertext Transfer Protocol Secure)

HTTPS is a secure extension of HTTP. It combines HTTP with an encryption protocol, either **TLS (Transport Layer Security)** or its predecessor, **SSL (Secure Sockets Layer)**. This provides a secure, encrypted channel for data transmission.

- **Usage**: It's the standard for secure communication, especially for websites that handle sensitive data, such as e-commerce sites, banking portals, and email services. All modern web browsers and servers support HTTPS.
- **Port**: HTTPS operates on port **443**.
- **Security**: The "S" stands for "Secure" because all data is **encrypted** before it's sent. This means even if the data is intercepted, it will be unreadable to the attacker. It also ensures **authentication**, verifying that you are connected to the correct server and not a fraudulent one, and **data integrity**, confirming that the data hasn't been tampered with during transit. The encryption process uses an **SSL/TLS certificate** to establish a secure connection, often indicated by a padlock icon in your browser's address bar.

b) DNS (Domain Name System):

The **Domain Name System (DNS)** is responsible for converting human-friendly hostnames (like *www.youtube.com*) into IP addresses that computers use to communicate. Without DNS, users would have to remember numerical IPs instead of names. In most cases, DNS communication runs over **UDP port 53**, since queries and responses are small and UDP introduces less overhead than TCP. TCP is still used in cases where reliability is required (e.g., large zone transfers or responses larger than 512 bytes).

A DNS message is built from a standard structure that applies to both queries and replies:

- **Transaction ID (16 bits):** A unique identifier that lets the client match each response with the query it sent.
- **Flags (16 bits):** Includes multiple sub-fields:
  - **QR (Query/Response):** Indicates whether the message is a request (0) or a reply (1).
  - **Opcode:** Defines the type of query (standard, inverse, or status).
  - **AA (Authoritative Answer):** Set when the responding server is authoritative for the queried domain.
  - **TC (Truncated):** Used if the message is too large for UDP (commonly set when response >512 bytes).

- **Counts:** The header contains four 16-bit fields indicating how many entries are present in the **Question**, **Answer**, **Authority**, and **Additional** sections.
- **Question Section:** Holds the domain name being resolved and the type of record requested (for example, A for IPv4 address, AAAA for IPv6, MX for mail servers, etc.).
- **Answer Section:** Contains the actual resource records with resolved data, such as the IP addresses corresponding to the queried domain.

In a normal lookup, the client sends a query packet containing the hostname and record type, and the DNS server replies with one or more answers. This lightweight exchange is what makes DNS fast and scalable across the internet.

DNS Header

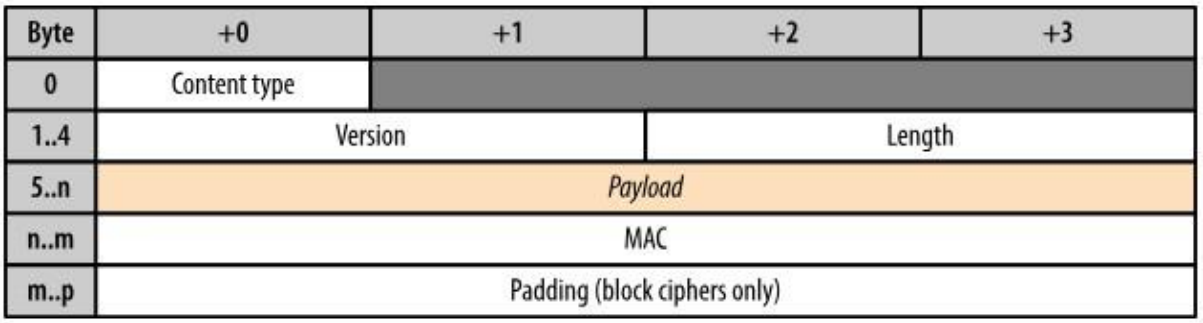
Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Transaction ID																QR	OPCODE				AA	TC	RD	RA	Z		AD	CD	RCODE		
4	32	Number of Questions																Number of Answers															
8	64	Number of Authority RRs																Number of additional RRs															

c) TLSv1.3 (Transport Layer Security):

Transport Layer Security (TLS) is the standard protocol that ensures secure, encrypted communication between clients (like web browsers) and servers. TLS 1.3 is the most recent version, streamlining the handshake process and improving both speed and security compared to earlier versions. A TLS record consists of a **header** followed by an **encrypted or plaintext payload**. The main header fields include:

- **Content Type:** Identifies what kind of TLS message is being transmitted, such as a handshake message (used for establishing keys), application data (the encrypted content), or alerts (used to indicate errors or warnings).
- **Version:** Specifies the TLS protocol version in use. Even though the record layer may sometimes display an older compatibility value, the actual handshake will negotiate TLS 1.3 features.
- **Length:** Indicates the size of the record, covering both header and payload. This helps the receiver parse messages correctly.
- **Payload:** Carries the actual message. During the handshake phase, this includes parameters like cipher suites and key exchange information. Once encryption is established, this section holds encrypted application data.
- **Authentication Data:** Each encrypted record includes integrity protection, ensuring that data cannot be tampered with while in transit.

With TLS 1.3, the handshake has been shortened to reduce latency, and outdated cryptographic algorithms have been removed. This makes it both faster and more resistant to modern attacks, while still preserving confidentiality and authenticity for web traffic like YouTube uploads.



Byte	+0	+1	+2	+3
0	Content type			
1..4	Version		Length	
5..n	*Payload*			
n..m	MAC			
m..p	Padding (block ciphers only)			

2) Transport Layer:

a) Transmission Control Protocol (TCP):

TCP is a reliable, connection-oriented protocol that ensures the accurate and orderly delivery of data between a client and a server. It breaks data into **segments**, numbers them, and reassembles them correctly at the destination. It's the foundation for many internet applications like web browsing.

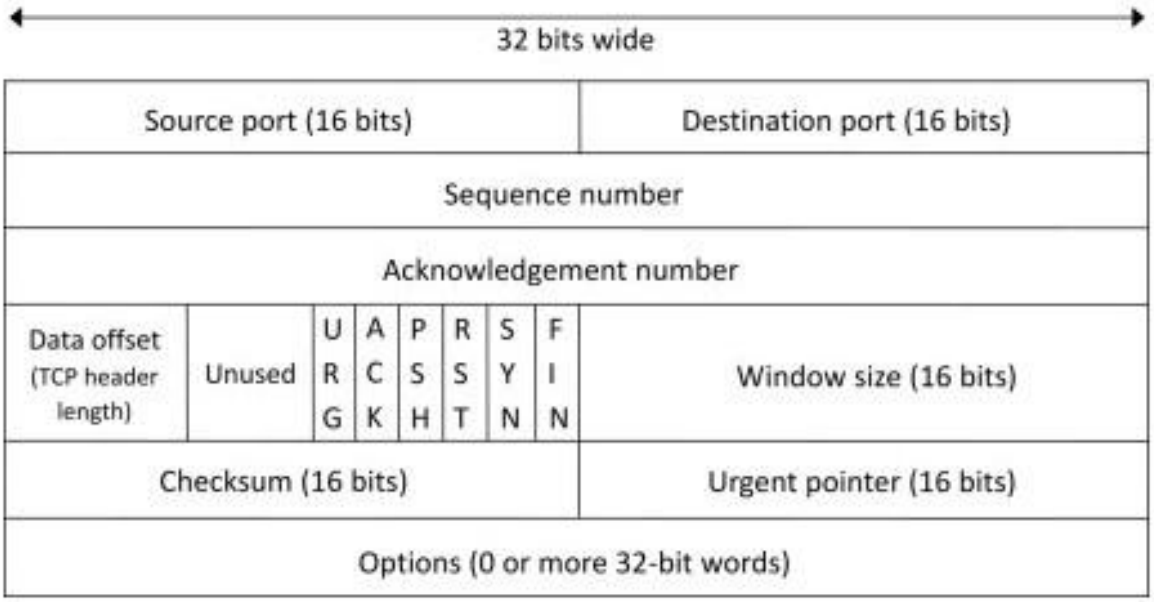
Key Fields in a TCP Header

- **Source and Destination Ports:** These 16-bit fields identify the specific applications on the sending and receiving devices, respectively.
- **Sequence Number:** A 32-bit field that helps **order segments** and reassemble data correctly.
- **Acknowledgement Number:** A 32-bit field used by the receiver to **confirm receipt** of data and request the next segment.
- **Control Flags:** Six 1-bit flags that control the connection state (e.g., SYN to initiate a connection, FIN to terminate it).
- **Window Size:** A 16-bit field used for **flow control**, preventing the sender from overwhelming the receiver.

How it Works: Three-Way Handshake

Before data transfer, TCP establishes a connection using a **three-way handshake** to ensure both sides are ready.

1. **SYN:** The client sends a segment to the server to request a connection.
2. **SYN-ACK:** The server responds, acknowledging the client's request and sending its own request.
3. **ACK:** The client sends a final acknowledgement, and the connection is established.



b) User Datagram Protocol (UDP):

UDP is a simple, connectionless protocol used for time-sensitive applications where speed is more important than perfect reliability, such as streaming video, online gaming, and voice over IP (VoIP). Unlike TCP, UDP doesn't establish a connection or guarantee delivery.

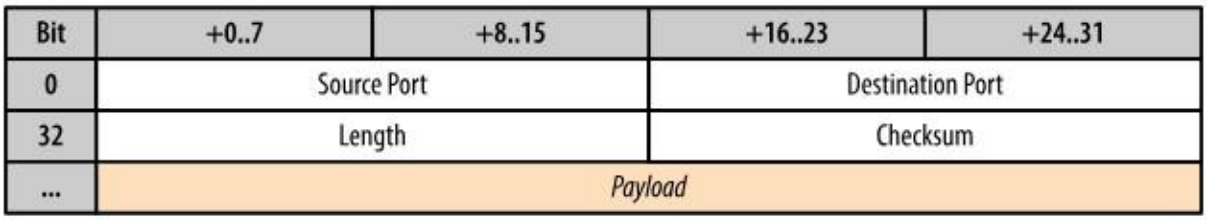
Key Fields in a UDP Header

- **Source Port**: A 16-bit field that identifies the port number of the originating application on the sender's device.
- **Destination Port**: A 16-bit field that identifies the port number of the receiving application on the destination device.
- **Length**: This 16-bit field specifies the entire length of the UDP datagram, including both the header and the user data.

- **Checksum**: A 16-bit field used for error checking. It allows the receiver to verify the integrity of the data and header. The checksum calculation is optional for IPv4 but mandatory for IPv6.

How it Works

UDP is often called a "fire-and-forget" protocol because it sends data without first establishing a connection. The sender simply transmits a **datagram** and doesn't wait for any acknowledgment of receipt. This makes it much faster and more efficient than TCP but also less reliable. If a datagram is lost or arrives out of order, UDP does not attempt to resend it or reorder the packets. This behavior is ideal for applications where a small amount of data loss is acceptable, as long as the data arrives quickly.



c) QUIC (Quick UDP Internet Connections)

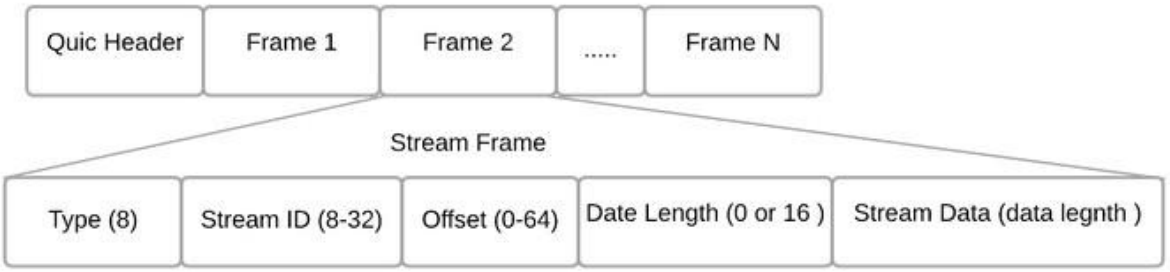
QUIC is a modern protocol developed by Google that runs over **UDP**. It's designed to improve web performance by combining the speed of UDP with the reliability of TCP, making it ideal for applications like Chrome and YouTube.

Key Features

- **Fast Connections:** QUIC establishes a secure connection in just one round-trip, much faster than TCP.
- **No Head-of-Line Blocking:** It allows multiple data streams to run independently, so a delay in one stream doesn't slow down others.
- **Reliable:** It includes built-in mechanisms for congestion control and retransmission to quickly recover from lost data.

Packet Structure

- **Header**: Contains a **Connection ID** for the unique connection and a **Packet Number** for sequencing.
- **Frames**: The payload is made of frames that carry different types of data, such as application data (**STREAM frames**) or acknowledgements (**ACK frames**).



3) Network Layer:

a) Internet Protocol Version 4:

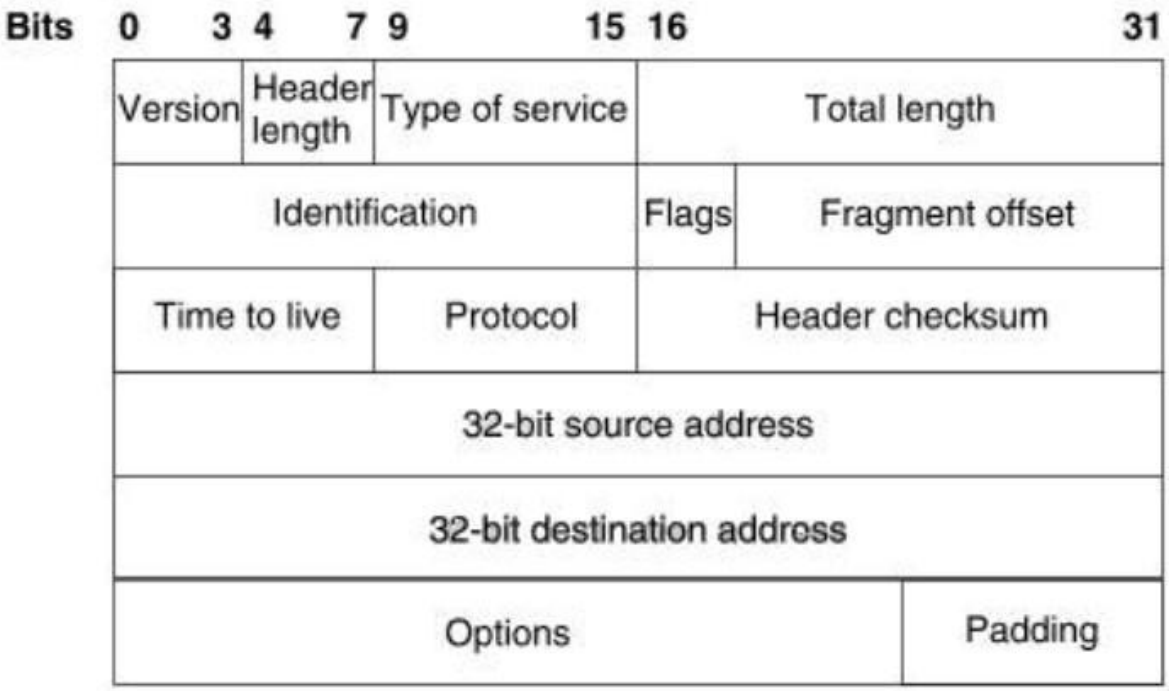
Internet Protocol version 4 (IPv4) is the fundamental protocol used to route data across the internet and other packet-switched networks. It defines the structure of data packets and the addressing system that allows them to be delivered from a source to a destination.

Key Fields in an IPv4 Header

- **Version**: A **4-bit** field that always contains the value **4**, identifying the protocol as IPv4.
- **Header Length**: This **4-bit** field specifies the size of the IPv4 header in 32-bit words. The minimum value is **5** (20 bytes).
- **Differentiated Services Code Point (DSCP)**: An **8-bit** field used to classify packets for quality-of-service (QoS) purposes, allowing for different levels of priority for specific types of traffic (e.g., real-time video vs. web browsing).
- **Total Length**: A **16-bit** field that indicates the total size of the IP packet, including both the header and the data. The maximum ()size is 65,535 bytes.
- **Identification**: A **16-bit** field that acts as a unique identifier for a packet. This is crucial for **fragmentation**, as it allows all fragments

of a single original packet to be reassembled correctly at the destination.

- **Flags**: A **3-bit** field that controls fragmentation. The "Don't Fragment" flag prevents a packet from being broken up, while the "More Fragments" flag indicates if more pieces of the packet are to follow.
- **Time to Live (TTL)**: An **8-bit** field that limits the lifespan of a packet. Its value is decremented by each router it passes through. If the TTL reaches zero, the packet is discarded, preventing it from endlessly looping on the network.
- **Protocol**: An **8-bit** field that identifies the next-level protocol (e.g., TCP, UDP) that the data payload belongs to.
- **Header Checksum**: A **16-bit** field used for error detection. It is calculated and checked by each router to ensure the header hasn't been corrupted during transit.
- **Source Address**: A **32-bit** field that contains the IP address of the sender.
- **Destination Address**: A **32-bit** field that contains the IP address of the intended recipient.



b) IPv6 (Internet Protocol version 6)

IPv6 was developed to succeed IPv4, providing a significantly larger address space along with enhancements such as a streamlined packet header format and better support for optional features and extensions.

Packet Structure:

- **Header:** Contains several important fields:
  - **Version:** Indicates the IP protocol version. For IPv6, this value is always 6.
  - **Traffic Class (8 bits):** Functions similarly to the Type of Service (ToS) in IPv4. It helps routers and switches prioritize packets, especially under network congestion.
  - **Flow Label (20 bits):** Identifies packets that belong to the same data flow, such as video streaming or VoIP. This allows routers to efficiently process all packets within a flow without examining the payload content.
  - **Payload Length (16 bits):** Specifies the size of the packet's payload in bytes, with a maximum of 65,535 bytes. If the payload is larger, the value is set to zero, and the "Jumbo Payload" extension header is used instead.
  - **Next Header (8 bits):** Indicates the type of the subsequent header, which could be a transport layer protocol (like TCP or UDP) or another IPv6 extension header, such as routing or fragmentation. This serves a similar purpose as the Protocol field in IPv4.
  - **Hop Limit (8 bits):** Functions like the Time to Live (TTL) field in IPv4, specifying the maximum number of routers the packet can traverse. Each router reduces this value by one before forwarding.
  - **Source Address (128 bits):** The IPv6 address of the sender. Its 128-bit length allows for a much larger address space compared to IPv4.
  - **Destination Address (128 bits):** The IPv6 address of the recipient, also 128 bits long.
- **Payload:** Contains the encapsulated data from the transport layer, such as TCP or UDP segments.

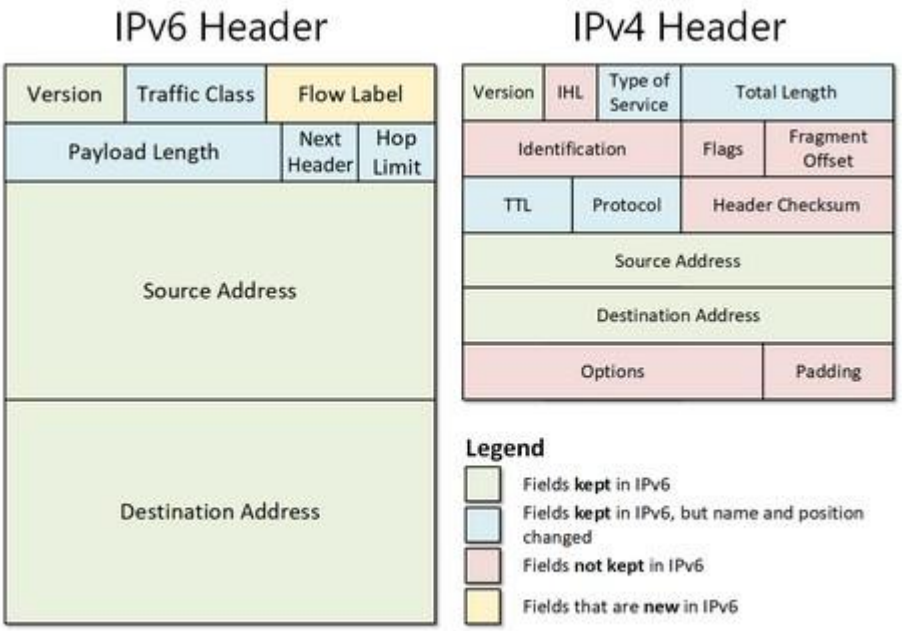


Figure 1. Comparing IPv4 and IPv6 headers

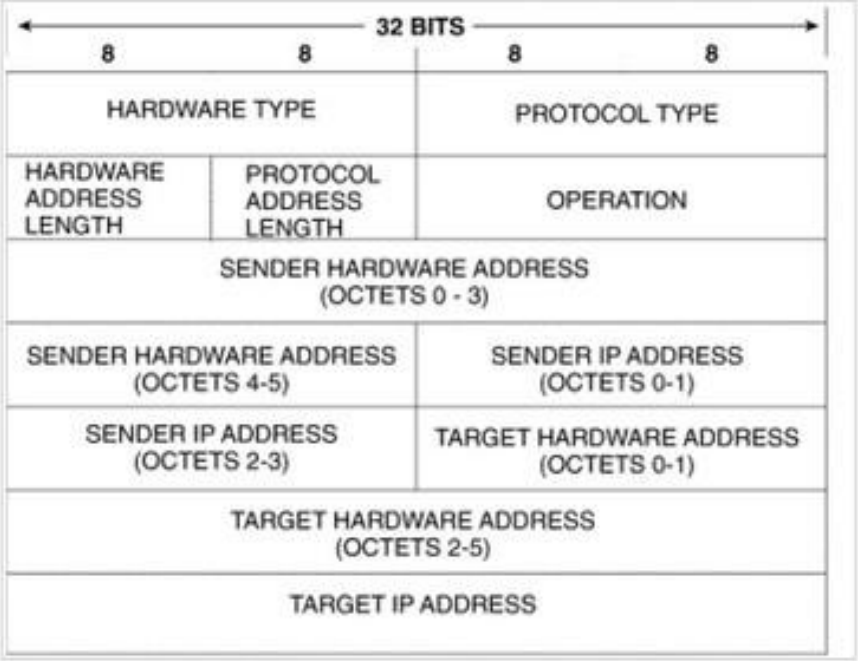
c) ARP (Address Resolution Protocol)

ARP (Address Resolution Protocol) is used to link a network layer IP address to a data link layer MAC address within a local network.

Packet Structure:

- **Hardware Type (16 bits):** Specifies the type of network hardware being used. For Ethernet networks, this value is 1, indicating the kind of interface in use.
- **Protocol Type (16 bits):** Defines the protocol whose addresses are being mapped. For IPv4, the value is 0x0800, indicating IP addresses are being resolved.
- **Hardware Address Length (8 bits):** Indicates the size of the hardware address in bytes. In Ethernet, this is 6 bytes since MAC addresses are 6 bytes long.
- **Protocol Address Length (8 bits):** Specifies the size of the protocol address in bytes. For IPv4, this is 4 bytes.
- **Operation (16 bits):** Shows the type of ARP message. A value of 1 represents an ARP request (asking for the MAC address associated with a given IP), and 2 represents an ARP reply (providing the requested MAC address).

- **Sender Hardware Address (variable length):** Contains the MAC address of the device sending the ARP packet. For requests, it's the requester's MAC; for replies, it's the responder's MAC.
- **Sender Protocol Address (variable length):** Contains the IP address of the sender.
- **Target Hardware Address (variable length):** Holds the MAC address of the intended recipient. In ARP requests, this is set to all zeros because the sender does not yet know the target's MAC. In replies, it contains the actual MAC of the target device.
- **Target Protocol Address (variable length):** Contains the IP address of the device the packet is intended for. This is included in both requests and replies.



4) Link Layer:

a) Ethernet II:

Destination Address: A 6-byte field that holds the MAC (Media Access Control) address of the device intended to receive the frame.

Source Address: A 6-byte field containing the MAC address of the device that sent the frame.
Type: Indicates the protocol of the data carried in the frame's payload, specifying which upper-layer protocol is encapsulated.

Question 2

Ethernet II

Destination MAC 6 Bytes	Source MAC 6 Bytes	Type 2 Bytes	Data 46 – 1500 Bytes	Frame Check Sequence 4 Bytes

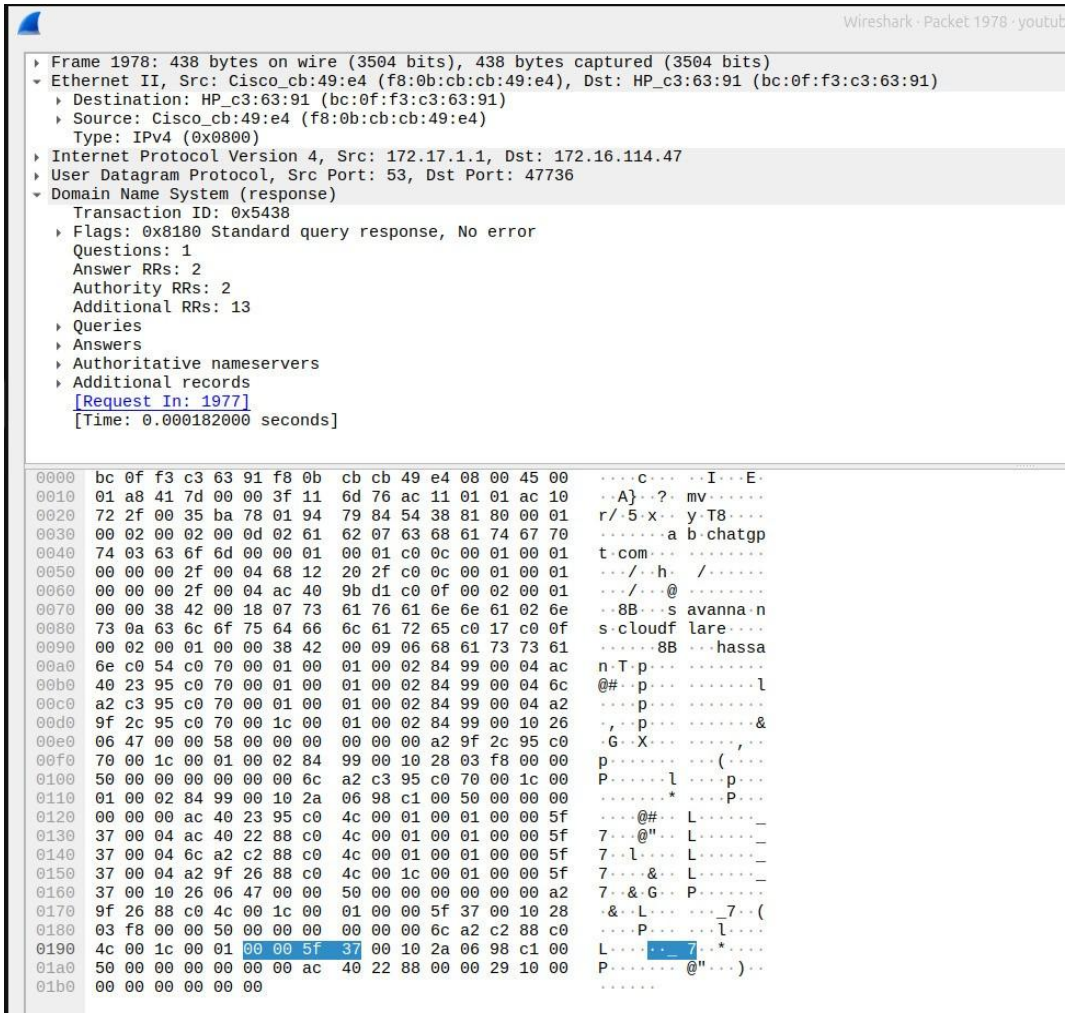
1) Application Layer:

a) HTTP/HTTPS (Hypertext Transfer Protocol / Hypertext Transfer Protocol Secure)



The Request Method is GET. The request version is HTTP/1.1

b) DNS (Domain Name System):



- The transaction ID of the DNS request is 0x5438 (16-bit).

- Response Flag tells us that this packet is a query and Opcode Flag specifies that the query type is standard.

- Questions in the pictures give the number of queries which is 1. Answer RR is 0, Authority RR is 2(meaning the server is pointing to 2 authoritative servers for this domain.) and Additional RR is also 13(server is including the IP addresses of several authoritative servers.).

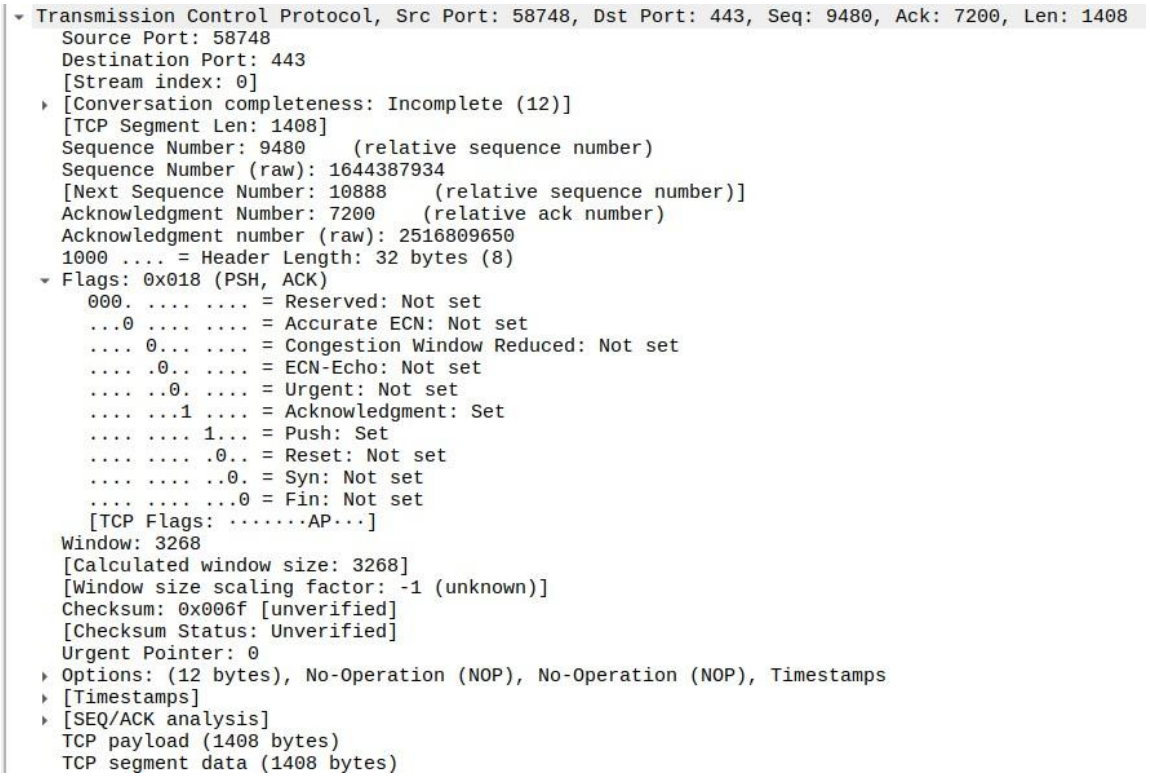
c) TLSv1.3 (Transport Layer Security)



We can observe that the TLS Version being used is TLS 1.0 and Content type is handshake. The TLS handshake protocol is being used to authenticate the participants of the communication and negotiate an encryption algorithm. The length of the packet including the header is 1894.

2) Transport Layer

a) Transmission Control Protocol (TCP)



Source/Destination Port: These numbers identify the specific applications on the sending and receiving devices, respectively.
Sequence Number (9480): A counter that tracks the bytes sent in this session, ensuring data is in the correct order.

Acknowledgment Number (7200): The number of the next byte the sender expects to receive from the other end. Since the ACK flag is set, this packet is acknowledging data.

Flags (ACK, PSH):

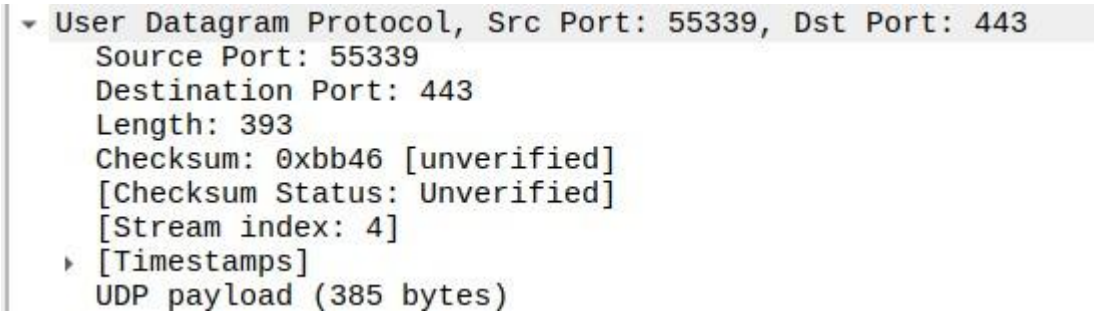
- ACK (Acknowledgment) flag: Confirms that the packet is acknowledging received data.
- PSH (Push) flag: Tells the receiving application to immediately process the data without waiting for a full buffer.

Header Length (32 bytes): The size of the TCP header. The standard is 20 bytes, so this packet includes 12 bytes of optional data (timestamps).

Window Size (3268): A flow control mechanism that tells the sender how many bytes the receiver is willing to accept before the sender must wait for an acknowledgment.

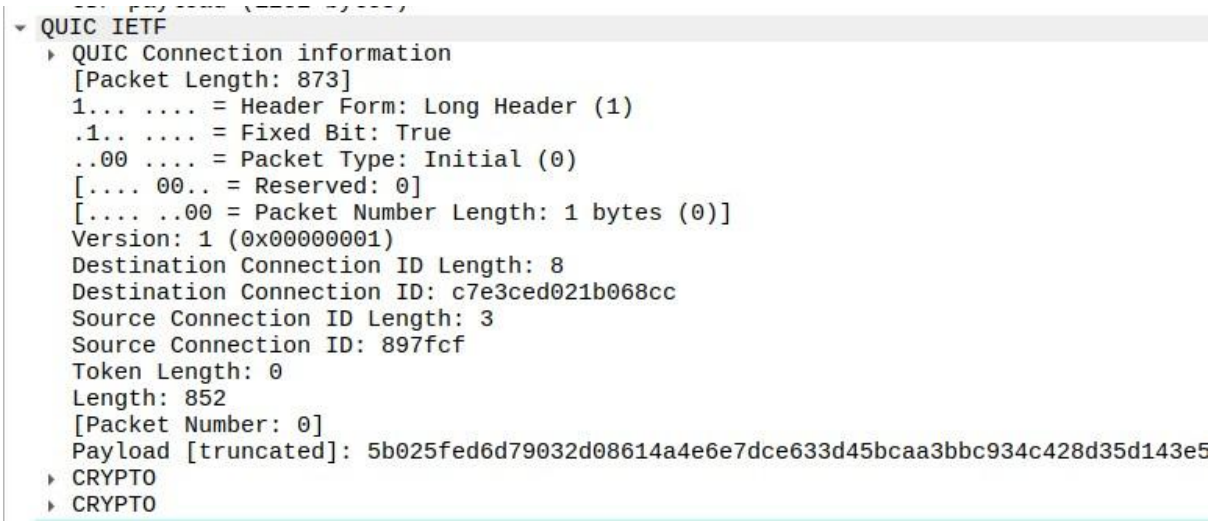
Urgent Pointer (0): Indicates that no urgent data is present in this packet.

b) User Datagram Protocol (UDP)



The Source Port and the Destination Port are 55339 and 443, respectively. The Length of the packet is 393 and the Checksum Status is unverified. Stream index displays a unique number for each stream, in this case it is 4, and is used to uniquely identify a TCP stream.

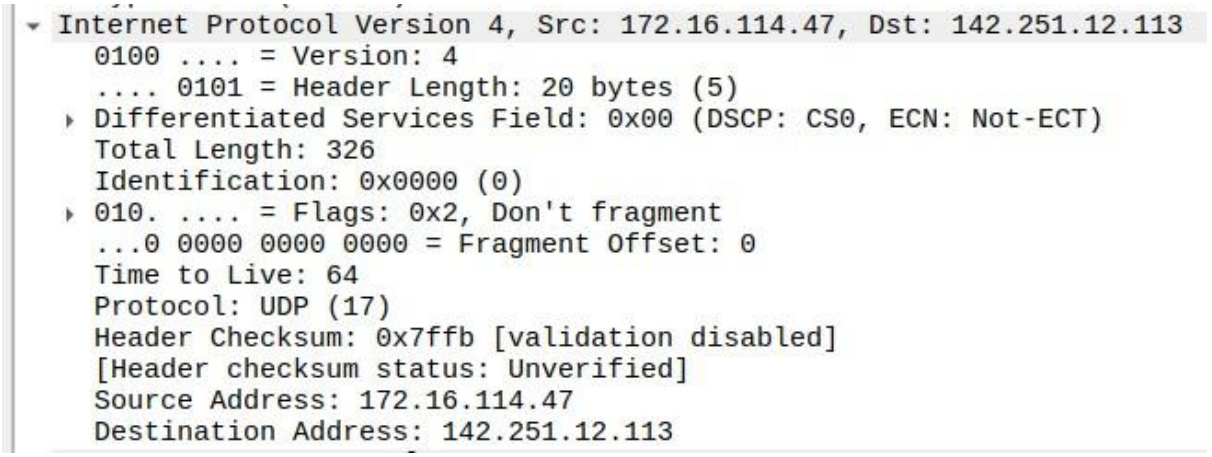
c) QUIC (Quick UDP Internet Connections)



The header form is 1 which signifies a long header. The token length is 0. The version is 1.

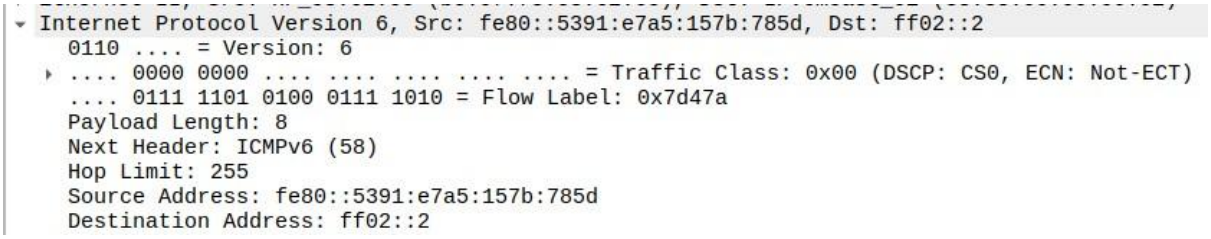
3) Network Protocol

a) Internet Protocol Version 4:



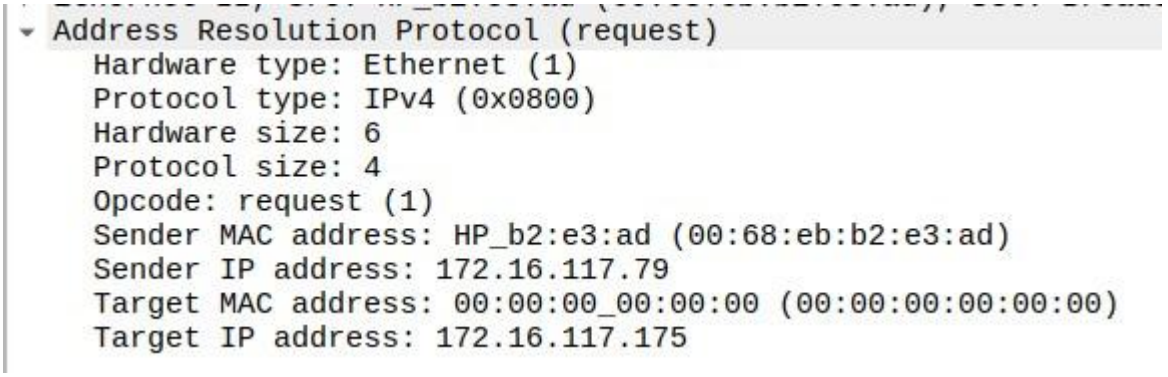
The version of IP protocol used is 4 i.e., IPv4. The Header Length is 20 bytes and the Total Length of the packet is 326. The Don't Fragment bit is not set which indicates that the packet can be fragmented. More Fragment bits are not set, so no fragments of the same packet will arrive later. Fragment Offset is 0, so this packet is not fragmented (because of the previous 2 flags). Time to Live is 64 which means that packet can make at most 64 hops. The next level protocol to be used is UDP and source and destination addresses are 172.16.114.47 and 142.251.12.113 respectively.

b) IPv6 (Internet Protocol version 6)



The version of IP protocol used is 6 i.e., IPv6. The next level protocol to be used is ICMPv6 and source and destination addresses are fe80::5391:e7a5:157b:785d and ff02::2 respectively.

c) ARP (Address Resolution Protocol)

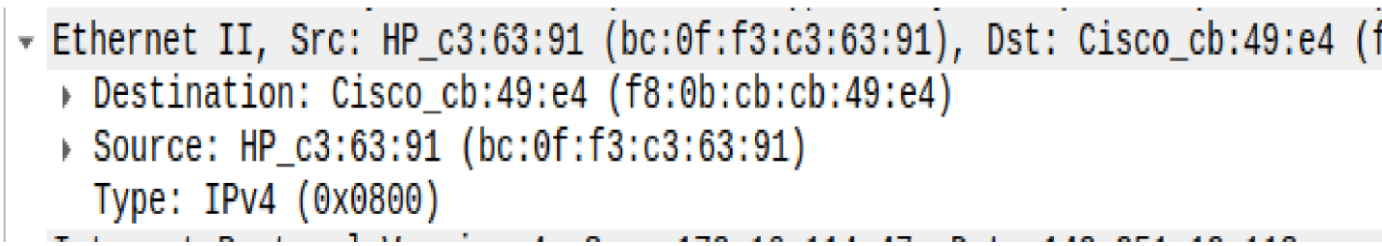


The hardware type is ethernet. The protocol type is IPv4, hardware size is 6, protocol size is 4. The opcode is request.

4) Link Layer:

a) Ethernet II:

The Source and Destination Values contain the source and destination device's MAC addresses which are HP_68:63:59 and Cisco_cb:49:e4 respectively. Type indicates the upper layer protocol used which is IPv4 in this case.



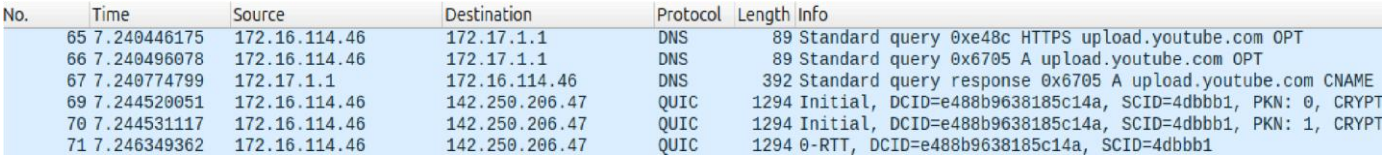
Question 3

Sequence of Messages Exchanged:

1) DNS Query/Response:

- The client resolves the domain name of YouTube (i.e. upload.youtube.com) to an IP address using DNS.

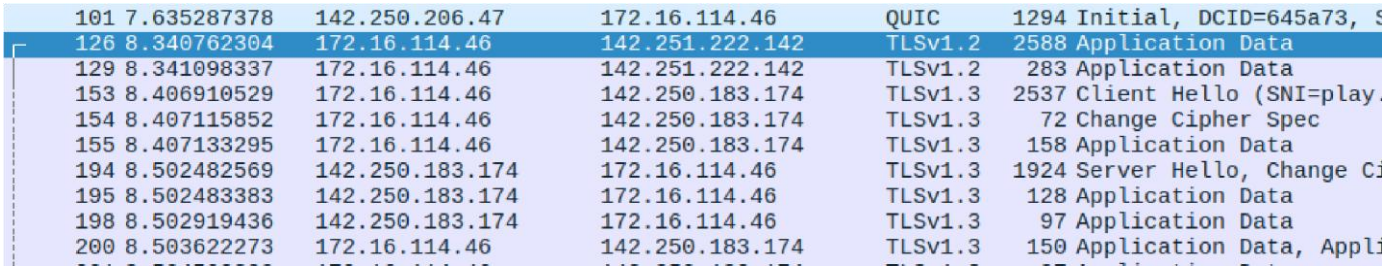
2) Handshake:



No.	Time	Source	Destination	Protocol	Length	Info
65	7.240446175	172.16.114.46	172.17.1.1	DNS	89	Standard query 0xe48c HTTPS upload.youtube.com OPT
66	7.240496078	172.16.114.46	172.17.1.1	DNS	89	Standard query 0x6705 A upload.youtube.com OPT
67	7.240774799	172.17.1.1	172.16.114.46	DNS	392	Standard query response 0x6705 A upload.youtube.com CNAME
69	7.244520051	172.16.114.46	142.250.206.47	QUIC	1294	Initial, DCID=e488b9638185c14a, SCID=4dbbb1, PKN: 0, CRYPT
70	7.244531117	172.16.114.46	142.250.206.47	QUIC	1294	Initial, DCID=e488b9638185c14a, SCID=4dbbb1, PKN: 1, CRYPT
71	7.246349362	172.16.114.46	142.250.206.47	QUIC	1294	0-RTT, DCID=e488b9638185c14a, SCID=4dbbb1

- QUIC Initial packet (includes TLS ClientHello), QUIC Handshake packet (TLS ServerHello), then encrypted traffic.

Client Hello: The client sends a Client Hello message to the server,



101	7.635287378	142.250.206.47	172.16.114.46	QUIC	1294	Initial, DCID=645a73, S
126	8.340762304	172.16.114.46	142.251.222.142	TLSv1.2	2588	Application Data
129	8.341098337	172.16.114.46	142.251.222.142	TLSv1.2	283	Application Data
153	8.406910529	172.16.114.46	142.250.183.174	TLSv1.3	2537	Client Hello (SNI=play
154	8.407115852	172.16.114.46	142.250.183.174	TLSv1.3	72	Change Cipher Spec
155	8.407133295	172.16.114.46	142.250.183.174	TLSv1.3	158	Application Data
194	8.502482569	142.250.183.174	172.16.114.46	TLSv1.3	1924	Server Hello, Change Ci
195	8.502483383	142.250.183.174	172.16.114.46	TLSv1.3	128	Application Data
198	8.502919436	142.250.183.174	172.16.114.46	TLSv1.3	97	Application Data
200	8.503622273	172.16.114.46	142.250.183.174	TLSv1.3	150	Application Data, Appli

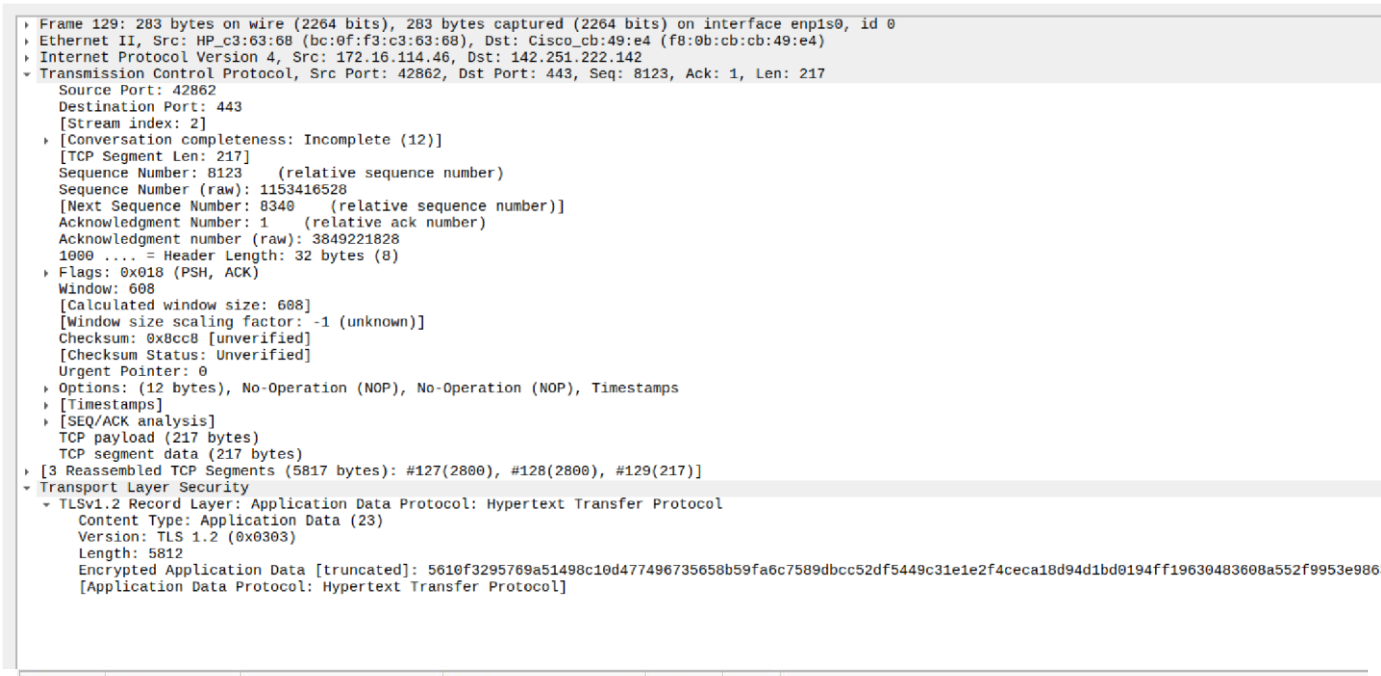
including supported cipher suites, TLS version, and a random number.

Server Hello: The server responds with a Server Hello, selecting the cipher suite, TLS version, and sending its digital certificate.

3) Initialising Uploading

Upload Request: The client sends an HTTP POST request to YouTube's upload endpoint, initiating the video upload process.
Server Response: The server responds with an upload URL and possibly an upload ID.

4) Uploading

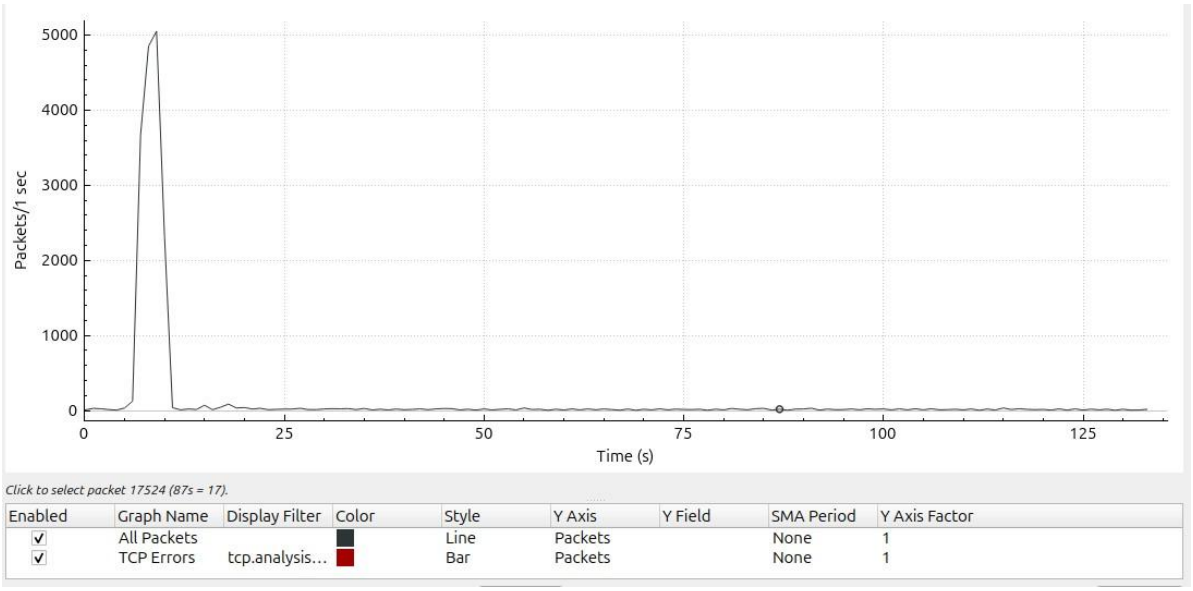


```
▸ Frame 129: 283 bytes on wire (2264 bits), 283 bytes captured (2264 bits) on interface enp1s0, id 0
▸ Ethernet II, Src: HP_c3:63:68 (bc:0f:f3:c3:63:68), Dst: Cisco_cb:49:e4 (f8:0b:cb:cb:49:e4)
▸ Internet Protocol Version 4, Src: 172.16.114.46, Dst: 142.251.222.142
▾ Transmission Control Protocol, Src Port: 42862, Dst Port: 443, Seq: 8123, Ack: 1, Len: 217
    Source Port: 42862
    Destination Port: 443
    [Stream index: 2]
  ▸ [Conversation completeness: Incomplete (12)]
    [TCP Segment Len: 217]
    Sequence Number: 8123    (relative sequence number)
    Sequence Number (raw): 1153416528
    [Next Sequence Number: 8340    (relative sequence number)]
    Acknowledgment Number: 1    (relative ack number)
    Acknowledgment number (raw): 3849221828
    1000 .... = Header Length: 32 bytes (8)
  ▸ Flags: 0x018 (PSH, ACK)
    Window: 608
    [Calculated window size: 608]
    [Window size scaling factor: -1 (unknown)]
    Checksum: 0x8cc8 [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  ▸ Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
  ▸ [Timestamps]
  ▸ [SEQ/ACK analysis]
    TCP payload (217 bytes)
    TCP segment data (217 bytes)
▸ [3 Reassembled TCP Segments (5817 bytes): #127(2800), #128(2800), #129(217)]
▾ Transport Layer Security
  ▾ TLSv1.2 Record Layer: Application Data Protocol: Hypertext Transfer Protocol
      Content Type: Application Data (23)
      Version: TLS 1.2 (0x0303)
      Length: 5812
      Encrypted Application Data [truncated]: 5610f3295769a51498c10d477496735658b59fa6c7589dbcc52df5449c31e1e2f4ceca18d94d1bd0194ff19630483608a552f9953e986
      [Application Data Protocol: Hypertext Transfer Protocol]
```

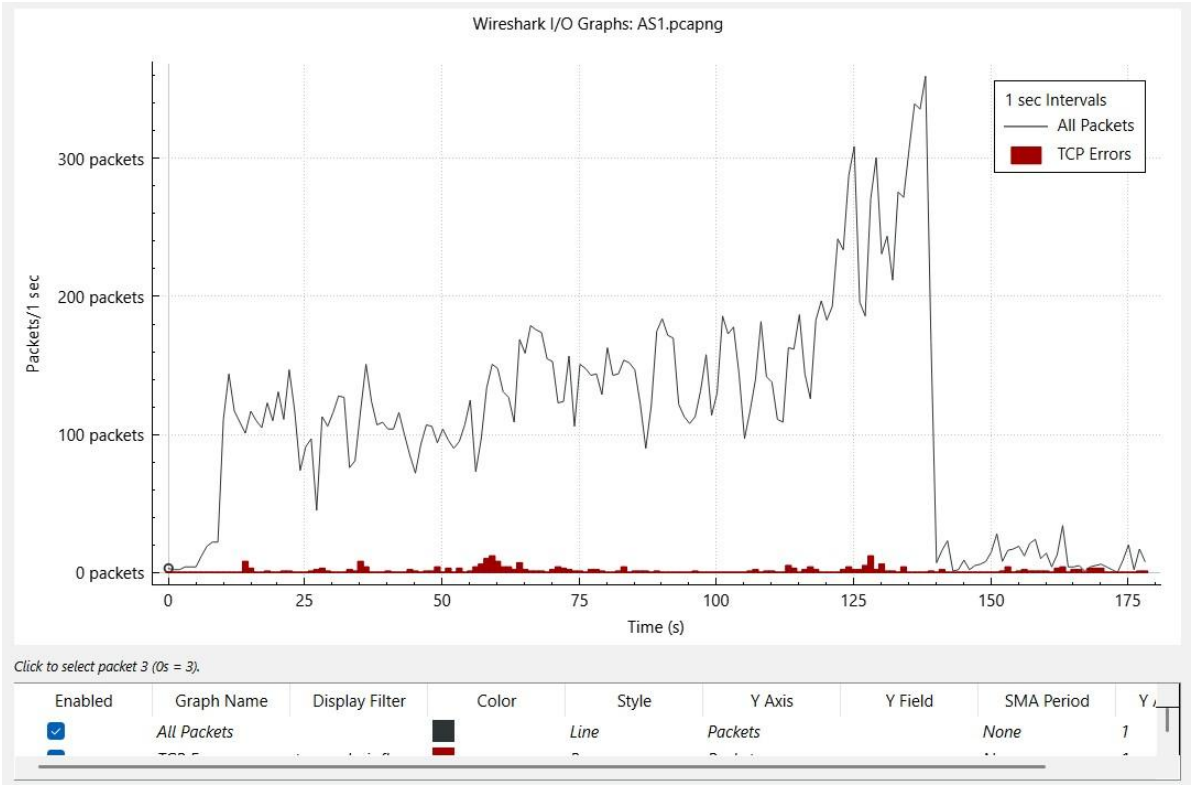
No.	Time	Source	Destination	Protocol	Length	Info
2100	9.176269582	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2101	9.176284465	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2102	9.176300112	172.16.114.46	142.250.206.47	QUIC	1181	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2104	9.208574834	142.250.206.47	172.16.114.46	QUIC	92	Protected Payload (KP0), DCID=645a73
2105	9.208962237	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2106	9.208976370	142.250.206.47	172.16.114.46	QUIC	92	Protected Payload (KP0), DCID=645a73
2107	9.209009355	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2108	9.209040625	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2109	9.209099188	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2110	9.209128863	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2111	9.209161014	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2112	9.209190030	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2113	9.209222698	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2114	9.209253980	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2115	9.209282888	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2116	9.209463835	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2117	9.209495476	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2118	9.209524767	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2119	9.209553816	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2120	9.209582744	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2121	9.209647739	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2122	9.209754765	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2123	9.209857172	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2124	9.209962435	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2125	9.210067991	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2126	9.212776010	142.250.206.47	172.16.114.46	QUIC	92	Protected Payload (KP0), DCID=645a73
2127	9.212819637	142.250.206.47	172.16.114.46	QUIC	92	Protected Payload (KP0), DCID=645a73
2128	9.213351986	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2129	9.213389199	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2130	9.213408582	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2131	9.213424930	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2132	9.213443423	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2133	9.213459552	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2134	9.213475416	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2135	9.213490931	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2136	9.213509662	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2137	9.213526453	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2138	9.213542032	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2139	9.213606225	142.250.206.47	172.16.114.46	QUIC	92	Protected Payload (KP0), DCID=645a73
2140	9.213640251	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2141	9.213728150	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9
2142	9.213832866	172.16.114.46	142.250.206.47	QUIC	1294	Protected Payload (KP0), DCID=fc9d0a83745e31a9

- The video file is divided into multiple binary chunks, each sent via POST.
- Server responds with 200 OK or 308 Resume Incomplete for partially received chunks.

In lab using wifi

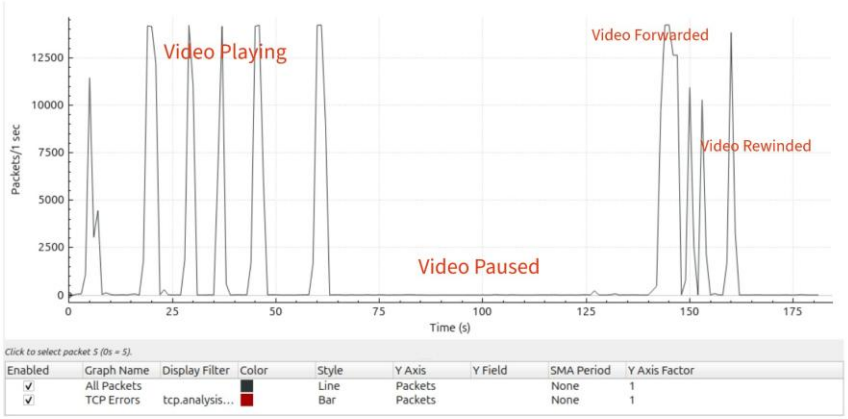


In hostel using mobile hotspot



5) Finalization:

- Client notifies upload complete, server responds with video ID + status.



Question 4

Secure and Efficient Video Uploads: A Protocol-Based Breakdown

Uploading a video to YouTube is a seamless process thanks to a suite of interconnected protocols that prioritize security, speed, and reliability. Each protocol plays a distinct yet crucial role in ensuring the video file reaches its destination intact and without compromise.

1. HTTPS: The Secure Foundation

- HTTPS (Hypertext Transfer Protocol Secure) is the primary protocol governing the secure transmission of all data during the upload.
- It establishes a secure channel between your device and YouTube's servers, ensuring that everything from the video file itself to sensitive user data like login tokens is encrypted.
- This encryption thwarts unauthorized access and protects against data manipulation by third parties.

- HTTPS also verifies the server's identity, guaranteeing you're uploading to the authentic YouTube service.

2. TLS: The Encryption Engine

- TLS (Transport Layer Security) is the underlying security layer that makes HTTPS possible.
- It's responsible for the core encryption, authentication, and data integrity checks.
- TLS creates a secure session by negotiating encryption keys, which are then used to protect the upload from prying eyes.
- For lengthy or interrupted uploads, TLS can reuse these session keys, enabling quicker reconnections and a smoother user experience.

3. QUIC: A Modern Approach to Speed and Reliability

- YouTube often leverages QUIC (Quick UDP Internet Connections), a protocol developed by Google to enhance data transfer efficiency, particularly on unstable networks.
- QUIC combines the best features of TCP and TLS, speeding up the initial connection and handling packet loss more gracefully than traditional TCP.
- QUIC operates over UDP (User Datagram Protocol), which is known for its low overhead and speed.
- By using UDP, QUIC avoids the time-consuming handshake and retransmission process of TCP, making uploads faster and more resilient.
- Like HTTP/2, QUIC supports multiple data streams over a single connection, which prevents a single stalled stream from blocking others (known as "head-of-line blocking"), thus improving overall upload efficiency.

4. TCP & HTTP/2: Reliable and Optimized Transport

- TCP (Transmission Control Protocol) is the traditional transport layer that underlies protocols like HTTPS and HTTP/2.
- Its primary function is to guarantee the reliable delivery of data packets.

- TCP ensures that all packets are received in the correct order, without errors, and it automatically retransmits any that are lost.
- It also manages data flow to prevent network congestion, which helps maintain a steady upload speed.
- HTTP/2, an evolution of the traditional HTTP protocol, is often used alongside TCP for its efficiency.
- It allows for the simultaneous transfer of multiple data streams—such as the video file, metadata, and thumbnails—over a single connection.
- This reduces the time it takes to complete the upload and minimizes the overhead of repetitive data in headers, which is especially beneficial for large files.

5. DNS: The Internet's Address Book

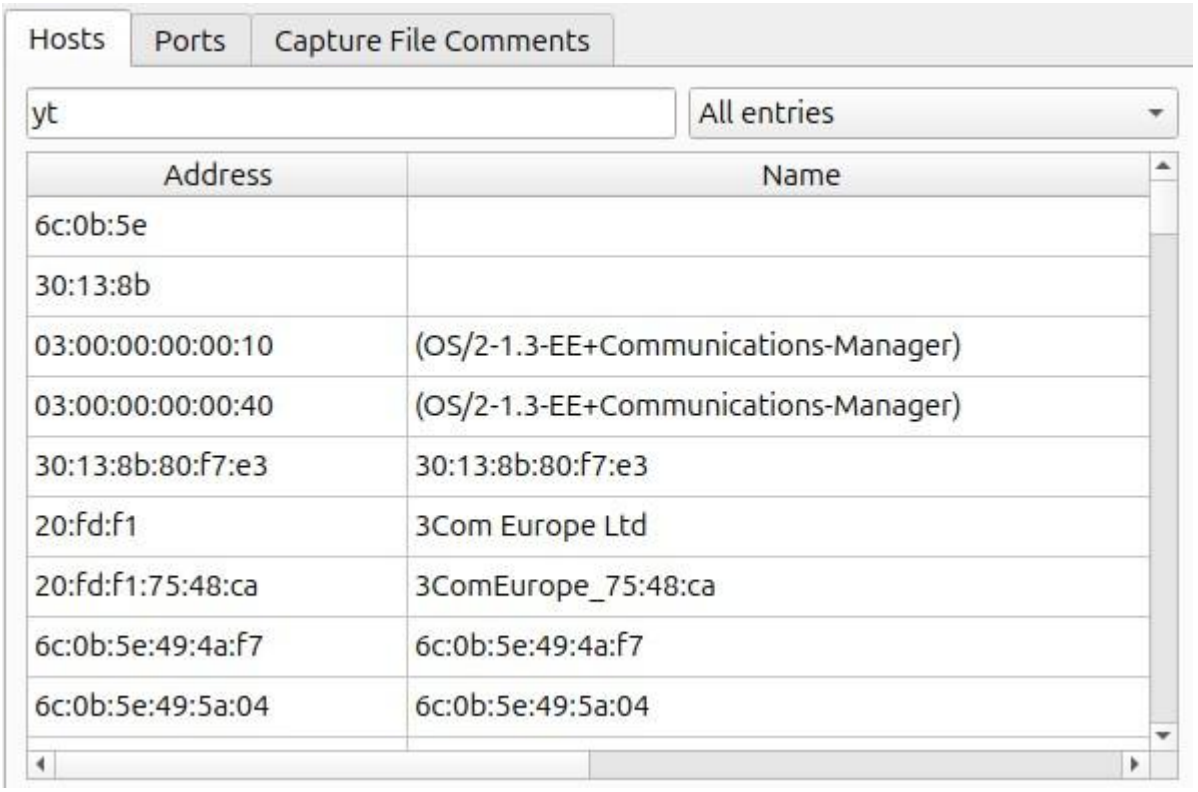
- Before any data can be transferred, DNS (Domain Name System) translates human-readable domain names (e.g., youtube.com) into computer-friendly IP addresses.
- This process is essential for your device to find and connect to YouTube's servers.
- Additionally, DNS can be used for load balancing, directing upload requests to various servers to ensure no single server is overwhelmed, which further enhances upload reliability.

Question 5

Calculating the different statistics at different times while uploading the youtube video.

Statistic	Afternoon	Evening	Using Hotspot
Throughput (kilobytes/sec)	620	760	22
RTT(in ms)	35	20	48
Packet size(avg)	1035	1059	1007
Number of packets lost	2	0	117
Number of UDP packets	16497	16224	18632
Number of TCP packets	1416	1271	1061
Number of responses received with respect to one request sent	0.40	0.45	0.30

Question 6



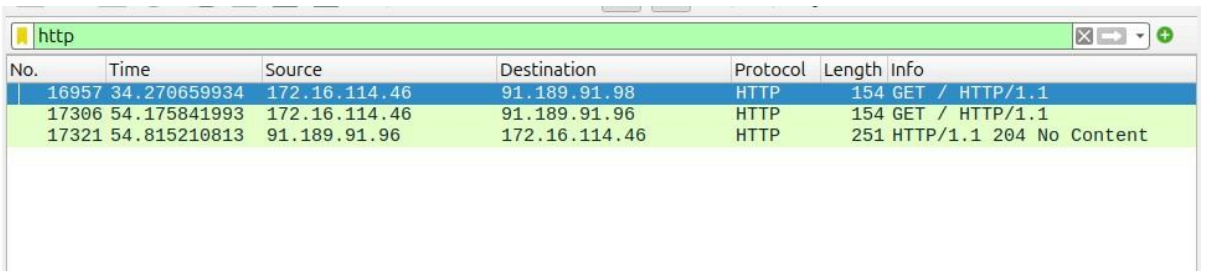
When analyzing network traffic, it's common to observe communication with multiple IP addresses for a single application. This distributed approach is a core strategy for modern web services to ensure efficiency and reliability. Here are the key reasons why this occurs:

1. **Content Delivery Networks (CDNs):** To minimize delays and improve user experience, applications utilize CDNs. These are globally distributed networks of servers that deliver content from the location closest to the user. This strategic placement ensures that data travels a shorter distance, which in turn reduces latency and boosts performance.
2. **Redundancy and Service Continuity:** By routing data through different servers, applications build in a layer of fault tolerance. This ensures that if one server fails or becomes overloaded, traffic can be seamlessly rerouted to another, guaranteeing uninterrupted service and continuous availability for the user.
3. **Traffic Distribution and Load Balancing:** To manage a high volume of requests, traffic is dispersed among several servers. This

practice, known as load balancing, prevents any single server from becoming a bottleneck, thereby improving overall system reliability and scalability. Consequently, network traces show connections to various IP addresses.

4. **Microservices Architecture:** Many large-scale applications are built using a microservices model, where different functionalities are handled by independent, specialized services. Each of these services can be hosted on a separate server, which results in network communication with multiple distinct IP addresses.
5. **Diverse Data Sources:** Applications may source data from various locations, including internal databases, third-party APIs, or external storage services. Each of these data sources is associated with its own IP address, which contributes to the multiple IPs seen in network traces.

Question 7

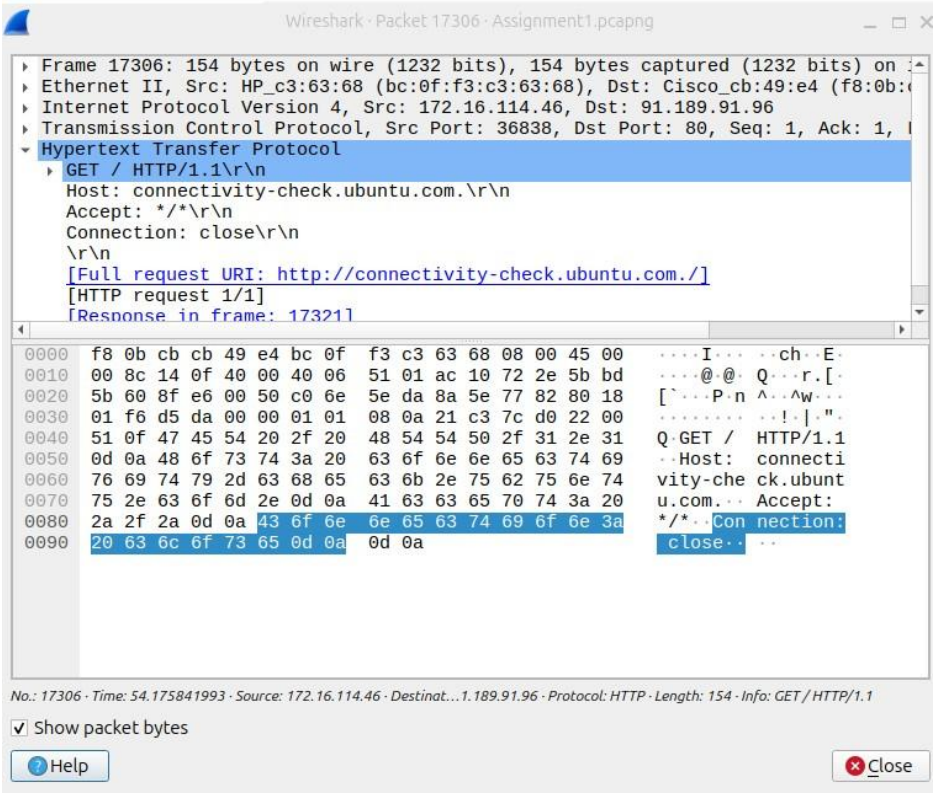


Server IP: 91.189.91.96

Client IP: 172.16.114.46

Browser Used: Firefox

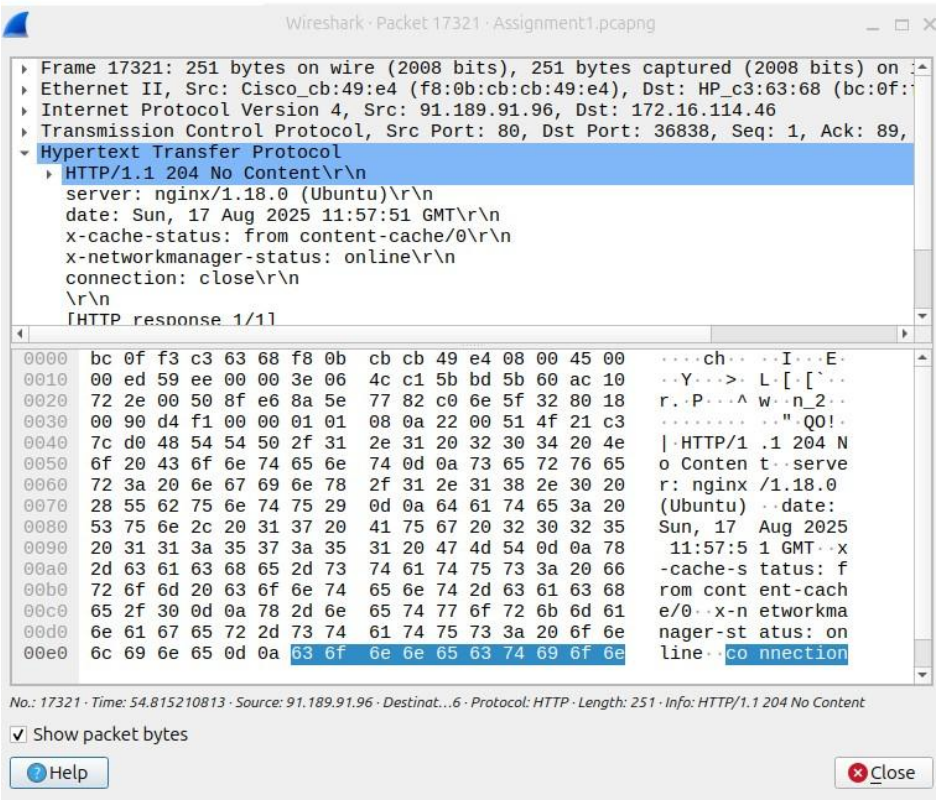
HTTP GET Request



Time: 54.17581993

Destination Port: 80

HTTP Response

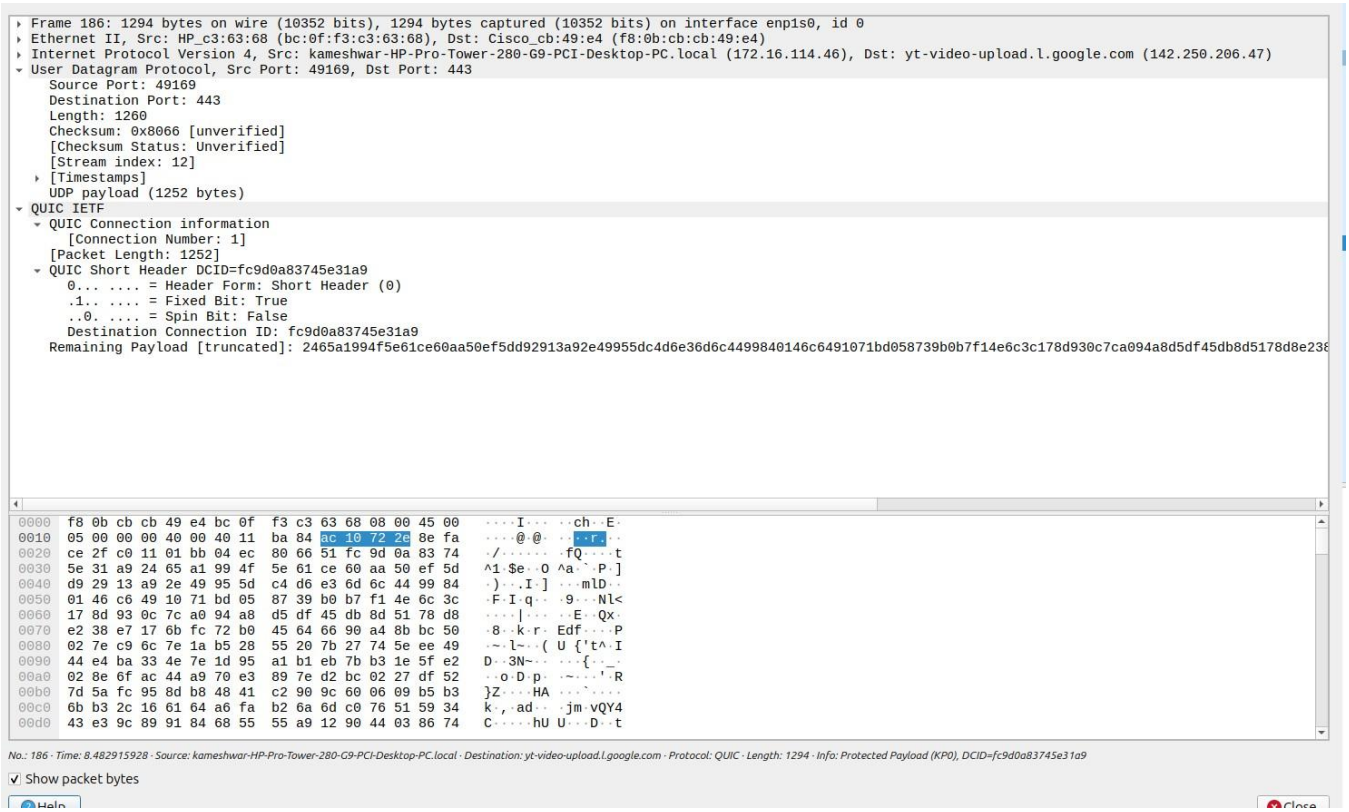


Time: 54.815210813

Destination Port: 36838

Difference: 0.639390883

HTTP POST(Encryption due to QUIC protocol)



Request URL: yt-video-upload.l.google.com
Destination Port: 443
Length: 1260

Question 8

```
annigandla-kameshwara-rao@kameshwar-HP-Pro-Tower-280-G9-PCI-Desktop-PC:~$ nslookup www.iitg.ac.in
Server:         127.0.0.53
Address:        127.0.0.53#53

Non-authoritative answer:
Name:   www.iitg.ac.in
Address: 172.17.0.22
```

IP Address of iitg.ac.in: 172.17.0.22

283	8.436768779	172.16.114.46	172.17.1.1	DNS	85	Standard query 0x5cdb HTTPS www.iitg.ac.in OPT
284	8.437037254	172.17.1.1	172.16.114.46	DNS	135	Standard query response 0x5cdb HTTPS www.iitg.ac.in
285	8.546615265	172.16.114.46	172.17.0.22	TCP	74	39520 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SA
286	8.546790941	172.17.0.22	172.16.114.46	TCP	74	443 → 39520 [SYN, ACK] Seq=0 Ack=1 Win=31856 Len=0
287	8.546828716	172.16.114.46	172.17.0.22	TCP	66	39520 → 443 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval
288	8.547342060	172.16.114.46	172.17.0.22	TLSv1.3	1965	Client Hello (SNI=www.iitg.ac.in)
289	8.547668599	172.17.0.22	172.16.114.46	TCP	66	443 → 39520 [ACK] Seq=1 Ack=1900 Win=31872 Len=0 TS

DNS Server IP Address: 172.17.1.1

DNS	507	Standard query response 0x53fc A www.aljazeera.com CNAME www.aljazeera.com.edg
DNS	85	Standard query 0x5cdb HTTPS www.iitg.ac.in OPT
DNS	135	Standard query response 0x5cdb HTTPS www.iitg.ac.in SOA dns2.iitg.ac.in OPT
TCP	74	39520 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2587993416 TS
TCP	74	443 → 39520 [SYN, ACK] Seq=0 Ack=1 Win=31856 Len=0 MSS=1460 SACK_PERM TSval=14
TCP	66	39520 → 443 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2587993417 TSecr=145213339

Name Server : dns2.iitg.ac.in

Wireshark · Packet 284 · enp1s0

```
▸ Frame 284: 135 bytes on wire (1080 bits), 135 bytes captured (1080 bits) on interface enp1s0, id 0
▸ Ethernet II, Src: Cisco_cb:49:e4 (f8:0b:cb:cb:49:e4), Dst: HP_c3:63:68 (bc:0f:f3:c3:63:68)
▸ Internet Protocol Version 4, Src: 172.17.1.1, Dst: 172.16.114.46
▾ User Datagram Protocol, Src Port: 53, Dst Port: 55901
    Source Port: 53
    Destination Port: 55901
    Length: 101
    Checksum: 0x6d0d [unverified]
    [Checksum Status: Unverified]
    [Stream index: 58]
  ▸ [Timestamps]
    UDP payload (93 bytes)
▾ Domain Name System (response)
    Transaction ID: 0x5cdb
  ▸ Flags: 0x8580 Standard query response, No error
    Questions: 1
    Answer RRs: 0
    Authority RRs: 1
    Additional RRs: 1
  ▸ Queries
  ▸ Authoritative nameservers
  ▸ Additional records
    [Request In: 283]
    [Time: 0.000268475 seconds]
```

Transport Protocol Used: UDP
Source Port (NS) : 53
Destination Port (Client) : 55901

Number of questions: 1
Number of answers: 0 DNS
caching:

- First lookup: query sent.
- Subsequent access: no query observed → caching at browser/OS level.

